Mechanistic Model Merging

Plain-language overview, project plan, evidence so far, and current checkpoint

Gavin + Codex

May 20, 2026

One-Slide Summary

- **Model merging** combines trained model weights or fine-tuning deltas into one model, often without the original training data.
- **Our project** asks why merging works or fails mechanistically, using the same discipline as the value-action project: prove the basis first, then interpret.
- **Current case** is a public Qwen2.5-1.5B merge where an abliterated TIES model loses harmful-refusal behavior while keeping benign behavior.
- **Main result today:** restoring a low-rank MLP activation-delta subspace over layers 12-24 repairs refusal behavior; strict manual audit makes `pca_rank64` the strongest compressed baseline.
- **Next step:** harden the refusal evaluator, validate on held-out prompts, then compare SAE/transcoder features against PCA rank 64.

What Is Model Merging?

A neural network is a large table of learned numbers, called **weights**.

Model merging means:

- start with two or more trained checkpoints;
- combine their weights, or the changes from a shared base model;
- produce one model that hopefully inherits useful behavior from each source.

It is done **before inference**. It is not asking several models and voting.

The Simplest Version

If two models start from the same base, a fine-tuned expert can be written as:

$$\theta_{\text{expert}} = \theta_{\text{base}} + \Delta_{\text{task}}$$

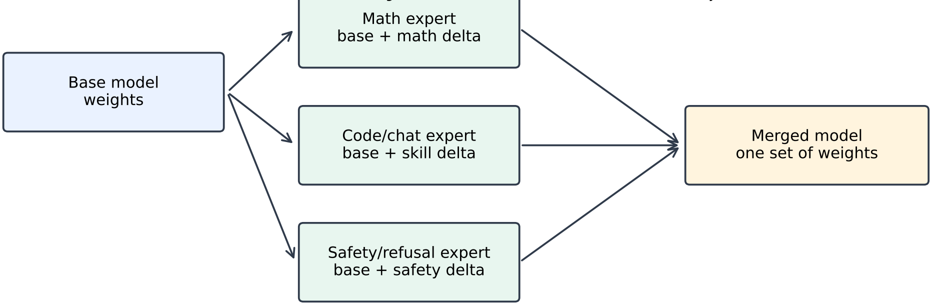
A simple merged model can be:

$$\theta_{\text{merge}} = \theta_{\text{base}} + \alpha\Delta_{\text{math}} + \beta\Delta_{\text{code}} + \gamma\Delta_{\text{safety}}$$

Here, the deltas are task vectors: directions in weight space created by fine-tuning.

Merging Is Not Concatenation

$$\theta_{merge} = \theta_{base} + \alpha\Delta_{math} + \beta\Delta_{code} + \gamma\Delta_{safety}$$



Merging changes the weights before inference. It is not concatenating models or voting between experts.

Why People Use It

Model merging is useful when:

- you have useful expert checkpoints but not the original training data;
- full joint training is too expensive;
- data cannot be shared because of privacy, license, or organization boundaries;
- you want one deployable model instead of many models at inference time;
- open-source communities want to combine math, code, chat, multilingual, or safety behavior.

Important: the claim is usually not “better than ideal full training.” The claim is often “better than what we can do with only checkpoints.”

Why It Is Hard To Understand

Merging can fail even when each source model is good.

- Two models may solve tasks using incompatible internal coordinates.
- Deltas may point in conflicting directions.
- A behavior may need multiple components: detection, policy, wording, formatting, and termination.
- A merge may preserve a keyword but lose the full mechanism.
- Metrics can over-credit shallow behavior, such as starting with a refusal and then giving unsafe advice.

Related Work Anchors

Work	What it does	Why it matters here
McMahan et al., FedAvg, AISTATS 2017	average client model updates	predecessor of checkpoint averaging
Izmailov et al., SWA, UAI 2018	average training checkpoints	averaging can find wider optima
Garipov et al., Mode Connectivity, NeurIPS 2018	low-loss paths between trained models	averaging works better when models are compatible
Wortsman et al., Model Soups, ICML 2022	average fine-tuned checkpoints	modern same-base merging result
Ilharco et al., Task Arithmetic, ICLR 2023	add/subtract task vectors	delta view of model editing
Yadav et al., TIES-Merging, NeurIPS 2023	trim and resolve sign conflicts	practical interference handling
Yu et al., DARE, ICML 2024	random delta dropping/rescaling	shows redundancy in deltas
Goddard et al., MergeKit, arXiv 2024	tooling for LLM merging	makes community merging practical

Citation counts are in the local project proposal; this slide is the conceptual map.

What We Want To Do

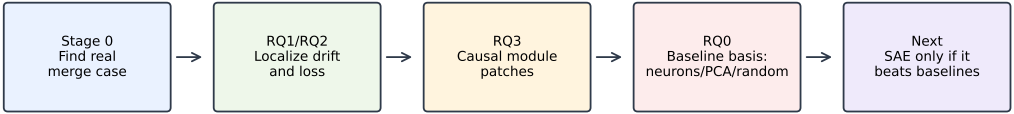
Central question:

When model merging works or fails, can we explain it in terms of identifiable mechanisms, features, modules, and causal pathways rather than only benchmark scores?

Near-term target:

- find a clean public-model merge failure;
- localize the missing behavior internally;
- repair it with causal interventions;
- then ask whether SAE/transcoder features explain more than raw activations, PCA, random projections, neurons, or module patching.

Project Pipeline



Principle imported from value_action: do not trust a sparse basis until it beats honest baselines.

Core Research Discipline

Borrowed from the value-action style:

1. Do not assume sparse features are meaningful because they look interpretable.
2. First test raw baselines: modules, activations, neurons, PCA, random projections, deltas.
3. Only use SAE/transcoder interpretation after a stable causal target exists.
4. If sparse features fail to beat simple baselines, report that honestly.

Research Questions: RQ0-RQ7

RQ	Question
RQ0	When does simple merging work, and which non-sparse baselines explain it?
RQ1	What predicts merge success before sparse features?
RQ2	Are inherited capabilities carried by the same modules as in the expert/base?
RQ3	Which modules are necessary and sufficient for inherited behavior?
RQ4	Why are some restored behaviors messy or low quality?
RQ5	What is merge interference mechanistically?
RQ6	Do expert deltas correspond to mechanisms?
RQ7	Are SAE/transcoder features a better basis than raw/PCA/random/module baselines?

Research Questions: RQ8-RQ15

RQ	Question
RQ8	What feature types exist during merging: inherited, lost, suppressed, conflicting, amplified, artifact?
RQ9	Can sparse feature patching recover missing capabilities?
RQ10	Can mechanism-aware merging outperform naive merging under constraints?
RQ11	Does the explanation generalize across models and tasks?
RQ12	How do standard algorithms differ mechanistically: linear, task arithmetic, TIES, DARE, DELLA, AdaMerging?
RQ13	Can merge failures be predicted before expensive evaluation?
RQ14	Does merging create genuinely new composition?
RQ15	What should practitioners do differently?

Experimental Phases

Phase	Goal	Current status
A	strengthen behavior benchmark	SmolLM2 taught us what not to trust
B	basis-validation benchmark	now active on Qwen2.5-1.5B
C	SAE/transcoder training/loading	gated, not started
D	sparse feature discovery	waiting for basis validation
E	causal feature validation	future
F	mechanism-aware merge recipes	future
G	generalization and paper claims	future

Stage 0: Controlled Sandbox

We first built small controlled cases:

- MNIST domain experts.
- SmolLM2-135M synthetic arithmetic, politeness, and refusal experts.
- Simple linear merges and alpha sweeps.

Result: simple merging can compose synthetic behaviors, but the setting was too artificial for a strong LLM paper by itself.

SmolLM2 Branch: Useful Failure

SmolLM2 taught an important lesson:

- refusal-like strings transferred;
- clean refusal almost did not;
- many outputs were artifacts, repetition, contradiction, or unsafe continuations;
- better synthetic refusal data improved things but did not produce a robust clean safety-transfer case.

Decision: do not spend expensive SAE work on that branch as the main clean-safety case.

Public Merge Screening

We screened public Qwen-family merges.

- Many 0.5B candidates were near-copies, weak, or behaviorally uninteresting.
- Qwen2.5-1.5B gave a stronger public case.
- Current model pair:
  - base: `Qwen/Qwen2.5-1.5B-Instruct`
  - failure: `nbeerbower/EVA-abliterated-TIES-Qwen2.5-1.5B`

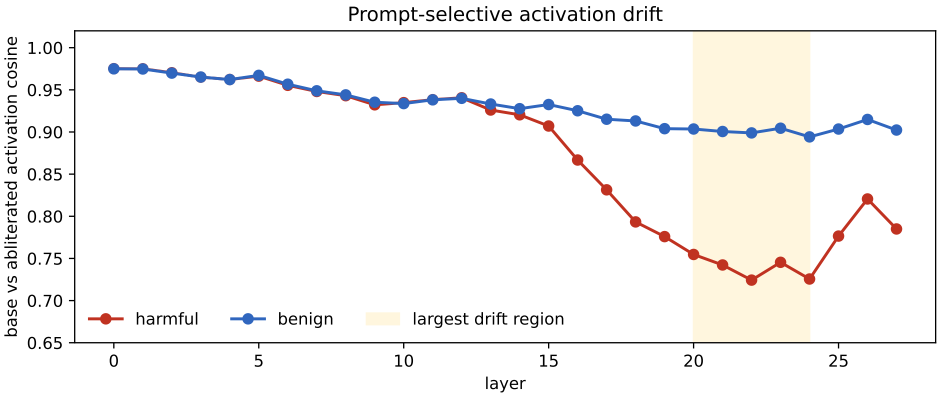
Why This Pair Is Good

The abliterated merge is a strong negative control:

- base refuses harmful prompts;
- abliterated model gives harmful instructions;
- benign helpfulness is mostly preserved;
- the weight delta is small enough that a mechanistic study is plausible;
- prompt-specific activation drift is visible.

This turns the project into a merge-induced safety-loss case study.

RQ1/RQ2: Activation Drift



Harmful prompts drift more than benign prompts in mid-to-late layers; layer 22 is especially different.

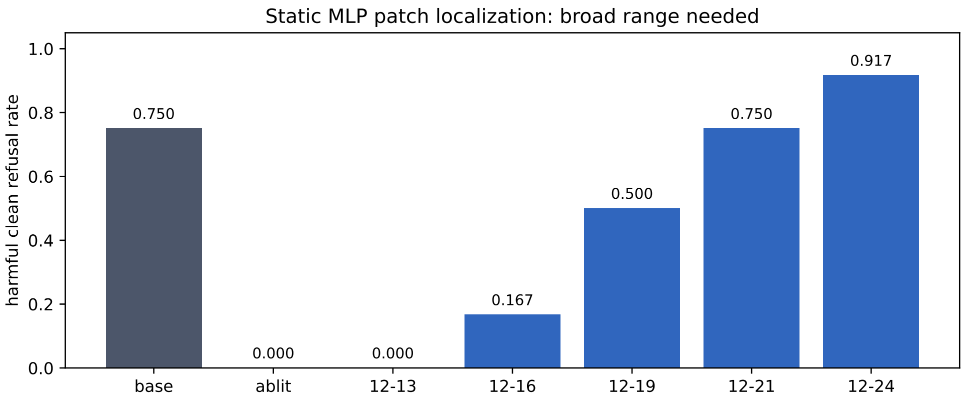
Single Modules Were Not Enough

Early target-loss patching pointed to mid-layer MLP/block modules.

But generation validation failed:

- `14:mlp`, `14:block`, `16:mlp`, and `16:block` did not restore harmful clean refusal.
- This separated likelihood localization from behavioral sufficiency.

RQ3: Broad MLP Range Is Causally Sufficient



Static base-to-abliterated MLP replacement needs a broad contiguous range. Layers 12-24 repair behavior best.

Activation Patch Confirms Mechanism Level

Teacher-forced activation patching over all positions:

- `12-24:mlp` closes 0.938 of harmful refusal-target loss gap.
- `12-21:mlp` closes 0.937.
- `12-19:mlp` closes 0.917.

Target-position-only activation patching closes 0.000 of the gap.

Interpretation: the repair is sequence-wide MLP activation propagation, not a single final-token vector.

Generation Patch Confirms Behavior

Dynamic activation patching runs the base donor on the current prefix and patches MLP activations into the abliterated recipient during generation.

Result on 8 harmful / 8 benign prompts:

- base harmful clean refusal: 0.875.
- abliterated harmful clean refusal: 0.000.
- dynamic `12-24:mlp` patch: 1.000.
- benign helpfulness remains 1.000; benign over-refusal remains 0.000.

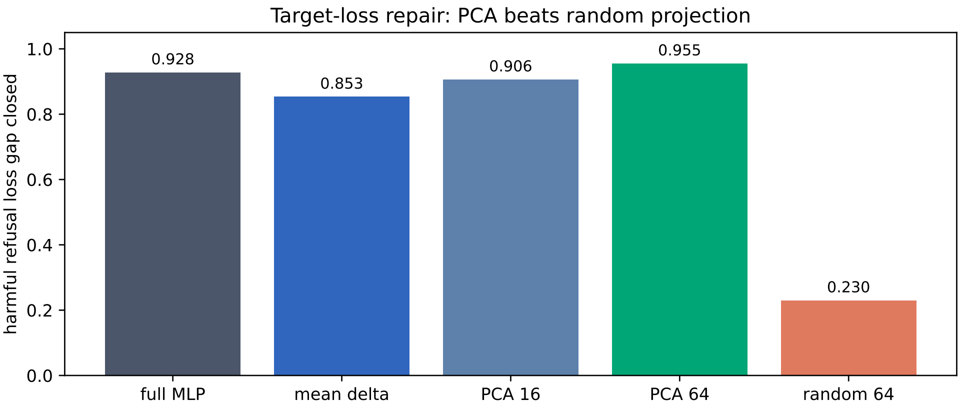
RQ0: Do Simple Bases Already Explain The Repair?

Before SAE/transcoder analysis, we tested compressed baselines:

- mean donor-recipient delta direction per layer;
- top changed neurons;
- harmful-vs-benign refusal direction;
- randomized PCA over activation deltas;
- matched random projection controls.

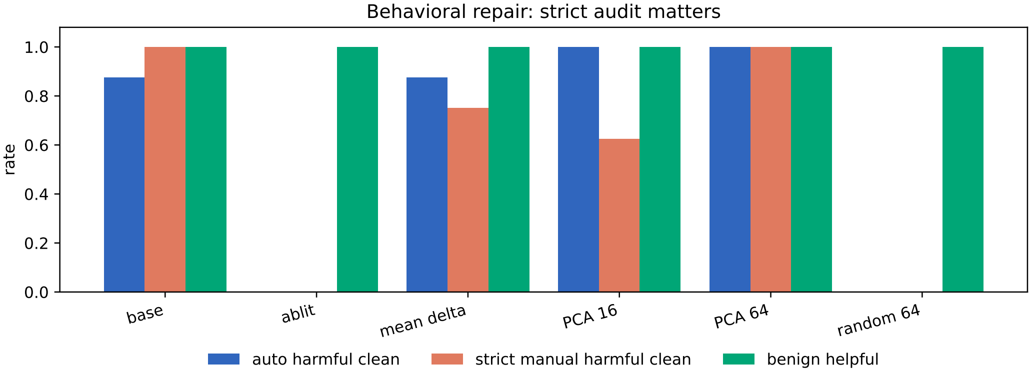
This is the key value-action style gate: sparse features must beat honest simpler bases.

Target-Loss Repair: PCA Wins



PCA rank 64 closes 0.955 of the harmful refusal-target loss gap. Matched random rank 64 closes only 0.230.

Behavior: Strict Audit Matters



Manual Audit Result

Model or patch	Auto harmful	Strict harmful	Benign helpful
Base	0.875	1.000	1.000
Abliterated	0.000	0.000	1.000
Mean delta rank 1	0.875	0.750	1.000
PCA rank 16	1.000	0.625	1.000
PCA rank 64	1.000	1.000	1.000
Random rank 64	0.000	0.000	1.000

The automatic scorer over-credited some answers that started with refusal but continued with unsafe advice. PCA rank 64 survived strict manual audit.

Current Mechanistic Interpretation

Best current hypothesis

The abliterated TIES merge loses a low-dimensional donor-recipient MLP activation-delta subspace across layers 12-24.

- Mean-delta rank 1 is a minimal partial repair.
- PCA rank 64 is the strongest compressed behavioral repair so far.
- Random rank 64 fails, so this is not just any 64-dimensional injection.
- SAE/transcoder features must explain, compress, or improve on this PCA subspace.

What Is New Here?

The project has moved from "can we find any model pair?" to a concrete causal object.

- We have a public model merge that loses a specific behavior.
- We can repair it by restoring a broad MLP activation pathway.
- We can compress that repair into a low-rank activation-delta subspace.
- We discovered a scoring failure mode that matters for safety claims.

What We Should Not Claim Yet

- Not a proof that model merging is generally safe or unsafe.
- Not a full safety benchmark; prompt sets are still small.
- Not an SAE result yet.
- Not a solved repair mechanism; PCA rank 64 is a strong baseline, not an explanation by itself.
- Not a claim that the abliterated merge is useful; it is a failure case we can study.

Immediate Next Steps

1. Fix the harmful-refusal evaluator so it detects unsafe continuation after refusal prefixes.
2. Build held-out harmful and adversarial benign prompt sets.
3. Revalidate `pca_rank64`, mean-delta, full MLP patch, and random controls.
4. Compare exact or larger-sample PCA to current randomized PCA.
5. Only then start SAE/transcoder feature experiments.

What SAE Must Beat

A useful sparse feature explanation should show at least one of:

- better repair than PCA rank 64 at similar or lower dimensionality;
- interpretable decomposition of the PCA repair into meaningful feature groups;
- causal feature patches that recover refusal without unsafe continuation;
- prediction of which merges will lose or preserve refusal before generation tests.

Paper Potential

Current status: continue, do not abandon.

Why:

- real public-model case;
- causal repair evidence;
- strong non-sparse baseline;
- clear evaluation caveat that can become part of the contribution;
- natural path to SAE/transcoder comparison.

Main risk: if sparse features do not beat PCA or add insight, the paper should be framed as a mechanistic low-rank activation-geometry study rather than an SAE paper.

Artifact Map

- Package: `presentations/2026-05-20-1618-model-merging-full-overview/`
- Proposal: `docs/MECHANISTIC_MODEL_MERGING_SAE_PROPOSAL.md`
- Provenance: `stage2/results/PROVENANCE.md`
- Main Qwen diagnostics: `stage2/results/qwen1_5b_refusal_rq1_rq2/`
- PCA/random target loss: `stage2/results/qwen1_5b_lowdim_activation_patch_target_loss_pca_random/`
- PCA/random generation: `stage2/results/qwen1_5b_lowdim_activation_patch_generation_pca_random/`
- Manual audit: `QWEN1_5B_LOWDIM_GENERATION_MANUAL_AUDIT.md`

Bottom Line

We now have a credible model-merging interpretability target:

A merge-induced refusal failure that can be repaired by restoring a low-rank MLP activation-delta subspace.

The next decisive question is whether sparse features explain this better than PCA rank 64.